AI TEXT WATERMARKING

Watermark My Words

When AI provenance starts changing the answer

OCTOBER 2026

Large language models are complicated systems built around a surprisingly simple idea: based on everything written so far, predict what should come next.

Modern models contain billions of parameters. They can read long documents, search, use tools, write code and do many things that do not look like autocomplete. But when they produce text, the basic process is still the same: choose the next piece of text, add it to the answer, and repeat. [2]

This matters because a new generation of AI text watermarking systems changes this process itself.

The idea behind watermarking is reasonable. If a text was generated by AI, it would be useful to have a way to detect that later. This is part of what is usually called **provenance**: knowing where content came from and, when possible, what happened to it along the way.

Images and videos have more options. A file can include **metadata**: extra information stored together with the content. This can include who created it, when it was created, which camera or software was used, or which tools were used to edit it. Systems such as Content Credentials can also add signed information about where the media came from and how it was changed. [8]

Text is different. A text file or document can also contain metadata, but when the text is copied into an email, a website or WhatsApp, this metadata is usually lost. What remains is only the text.

Because of this, some systems hide the watermark inside the way the model creates the text.

First, how does an LLM actually write?

Let's start with a simple example.

Give a language model this sentence:

```
The capital of France is
```

The model does not have one answer waiting inside it. It calculates how likely many possible continuations are.

A made-up example could look like this:

```
Paris       96%
Lyon         1%
Marseille    1%
the          0.5%
...          ...
```

This list of possible answers and their probabilities is called a **probability distribution**.

The numbers above are made up, but this is roughly the kind of calculation the model is doing.

The model then needs to choose one of these options. The rules used to make this choice are called **decoding** or **sampling**. Sometimes the most likely option is selected. Sometimes the system adds some randomness so the model does not produce the exact same answer every time.

In this example, "Paris" is much more likely than the other options.

Now look at a different sentence:

```
It was a beautiful
```

There are many reasonable ways to continue:

```
day
morning
place
sunset
moment
...
```

Here the model has much more freedom.

This freedom gives the watermark room to hide.

Words are not exactly words

I am using "next word" because it is easier to explain, but language models actually work with **tokens**.

A token is a piece of text. It can be a full word, part of a word, punctuation, or even part of a number. Before the model receives text, a **tokenizer** breaks it into these pieces.

Depending on the tokenizer, a word such as:

```
watermarking
```

may be split into pieces that look more like:

```
water + mark + ing
```

A number can also be split. Something like:

```
50mg
```

may become separate pieces for `50` and `mg`, or be split differently. The exact split depends on the model.

What matters is:

> *The model repeatedly decides which token comes next.*

After a token is selected, it becomes part of the text used to calculate the next token.

Generation therefore looks roughly like this:

```
The
The ferret
The ferret carried
The ferret carried a
The ferret carried a bazooka
```

The mechanism is the same whether the model is writing about a ferret with a bazooka, a legal contract or medical instructions.

At every step, the model looks at what came before and calculates a new set of probabilities. [2]

This gives us an important property of LLMs:

Changing one token can also change every decision that comes after it.

For example, if the model writes:

```
The meeting will take place...
```

the next token is calculated from different text than if it had written:

```
The session will take place...
```

To us, "meeting" and "session" may mean almost the same thing. To the model they are different inputs, so the probabilities for what comes next are different as well.

This becomes important when another system starts pushing these choices in a specific direction.

So where can we hide the watermark?

With images and videos, there are many places to hide extra information. Tiny changes can be made to pixels, and the file can also carry metadata.

Plain text gives us much less to work with.

Take this sentence:

```
Please send the report tomorrow.
```

Once it is copied as plain text, there are no hidden pixels and usually no useful metadata. There are only the characters.

One solution is to hide the watermark in **how the model chooses the text**.

A classic text-watermarking system uses a secret key. A secret key is simply a value known to the system that creates the watermark and to the system that later checks it.

The key is used to divide possible next tokens into groups. During generation, tokens from one group get a small advantage.

A normal reader just sees normal text. A detector that knows the secret can later check whether the text contains more of the preferred choices than we would expect by chance. This is the basic idea behind the well-known "green token" watermark proposed by Kirchenbauer and others. [6]

Imagine a model choosing between:

```
The weather is nice today.
The weather is pleasant today.
```

Both are fine.

If the watermark prefers "pleasant", choosing it does not really change the meaning. But it adds a tiny signal to the text.

Repeat this many times across a few hundred words and the detector may have enough information to say that the text probably came from the watermarked model.

Modern systems are more complicated than a secret list of preferred words.

OpenAI's new system, **textGrain**, divides possible tokens into secret groups based on a key and uses a mathematical process to influence which group is selected. Inside the selected group, it keeps the original relative probabilities between the tokens. It also limits how much freedom the watermark can take away from the normal sampling process. [2]

OpenAI's textGrain paper describes this limit using **entropy**. In this case, entropy is basically a measurement of how much choice the model has. If ten different words are all reasonable, entropy is high. If there is only one sensible answer, entropy is low.

OpenAI calls this limit an **entropy budget**. [2]

The math is more advanced than in the early watermarking systems, but for this discussion one thing matters:

> *Without the watermark, the token is selected from the model's normal probabilities. With the watermark, a secret key now has some influence over the selection.*

If the key had no influence, there would be no watermark to detect. The textGrain paper says this directly: if tokens are independent of the keyed randomness, there is no watermark signal. [2]

So where's the problem?

LLMs are already far from perfect. They hallucinate facts, misunderstand questions and sometimes produce complete nonsense with full confidence.

Watermarking now gives the generation process another job: while producing the answer, it also needs to leave enough of a secret statistical pattern for a detector to find later.

For normal prose, this may be harmless:

```
nice       -> pleasant
big        -> large
started    -> began
```

But not every token is decoration.

```
5 mg       -> 50 mg
1 tsp      -> 1 tbsp
May 12     -> May 21
do         -> do not
==         -> !=
```

These are very small changes in the text, but they can completely change the result. In medicine, some of these changes can be deadly.

textGrain does not keep a secret list saying "`50 mg` is better than `5 mg`". That is not how the system works.

But the watermark is still influencing the same token-selection process that decides whether the answer says `5`, `50`, `mg`, `g`, `do` or `do not`.

The extra influence is there to make the text detectable later. It is not there to make the answer more correct.

This is where the enshittification starts: an already imperfect system gets another requirement that can push the output away from what it would normally produce, so we can later prove where the text came from. The user gets no benefit from this change.

What does the research say about this?

A 2025 paper published in the Findings of EMNLP looked specifically at watermarking medical text. [4]

The researchers tested several watermarking systems and found more factual errors and problems with medical entities in the watermarked text. [4]

This matters because AI output can look completely fine while still being wrong. A sentence can sound professional, be grammatically correct and easy to read, but still be medically incorrect to the point of life or death.

This is not to say the research found a specific case where watermarking caused an LLM to recommend 50 mg instead of 5 mg, but it **does** show the danger: the text can still look normal while the medical information gets worse.

Then, in October 2026, another paper looked at the problem more directly.

The paper is called "Invisible Ink, Visible Lies: How Production Watermarking Causes LLMs to Hallucinate", and it was accepted at NeurIPS 2026. [3]

The researchers gave the model the information needed to answer factual questions and compared normal answers with watermarked answers using the same context, question and generation settings. [3]

The result that matters looked like this:

```
without watermark: correct
with watermark:    wrong
```

They tested six representative watermarking methods and found this type of failure in all six. [3]

If the model has the correct fact in front of it, gets it right without watermarking, and gets it wrong with watermarking, then the watermark made the answer worse. This is the exact kind of enshittification that general quality tests can miss.

The wrong token doesn't even need to be watermarked

The obvious fix is:

> *Don't watermark numbers, dates, drug names, units, URLs or other values where one character can matter.*

This is probably a good idea, but it does not solve the whole problem.

Remember how the model generates text:

```
token -> new context -> token -> new context -> token -> ...
```

Every token becomes part of the input used to generate the next one.

The NeurIPS paper found two ways this can happen. [3]

The first is simple: the watermark directly affects the choice when the factual token is generated.

The second starts earlier.

A watermark-driven change can alter the text before the fact. The model then calculates everything that follows from this slightly different text.

The researchers call this **prefix-induced attention drift**.

A **prefix** is the text already generated before the current token.

Attention is part of the mechanism the model uses to decide which parts of the available text are important for the next prediction.

In their analysis, a harmless-looking wording change earlier in the sentence changed how much attention the model later gave to the supplied factual information. [3]

By the time the model reached the actual fact, it was already following a different path.

So the watermark does not need to directly change the 12 in "May 12" to eventually produce "May 21". It may be enough to change something earlier.

This also means that a blacklist of numbers, units and other sensitive tokens is not enough. The mistake can start with a word that looked harmless.

Can we just watermark the "safe" words?

Another possible fix is to watermark only places where the model has many reasonable choices.

If the model is almost certain about the next token, leave it alone.

This approach already exists.

A watermarking method called **SWEET**, designed for code generation, uses token entropy to decide when to apply the watermark. Its authors introduced this specifically because ordinary watermarking performed badly on low-entropy code and could damage code quality. [5]

Again, entropy here means how uncertain the model is about the next token.

Consider:

```
The sky looked ___
```

There may be many good answers.

Now compare this with:

```
2 + 2 =
```

There should not be much room for creativity.

Code has the same problem. One quote mark, operator or number can be the difference between working software and broken software.

So avoiding low-entropy decisions makes sense.

The problem is that "the model is unsure" does not mean "this token is safe to mess with".

The model can be unsure between several dates, names or numbers even when only one of them is correct. Entropy tells us how much choice the model has. It does not tell us how important the choice is.

So this rule makes sense:

```
low uncertainty = be careful
```

This one does not:

```
high uncertainty = safe to change
```

A value can be uncertain for the model and still be very important.

But OpenAI says the quality stays the same

OpenAI reports that textGrain does not meaningfully reduce the scores of its Astra model on the benchmarks it tested. [1]

A **benchmark** is a standard set of tests used to measure how well a model performs at tasks such as answering questions, coding, browsing or medical reasoning.

The published results include:

Benchmark	Without watermark	With watermark
HealthBench Professional	64.27%	64.60%
BrowseComp	87.92%	87.35%
GPQA Diamond	94.44%	93.94%
DeepSWE v1.1	72.80%	71.68%

These numbers do not show a major drop. Some go slightly up and some slightly down. OpenAI says it does not see meaningful performance differences with and without watermarking. [1]

Google DeepMind reported similar results for its SynthID-Text watermark. Its production experiment compared roughly 20 million watermarked and unwatermarked Gemini responses and found no statistically significant difference in user feedback. [7]

So watermarking does not automatically make a model much worse.

But averages are very good at hiding a small number of ugly failures.

Imagine a model answers 100,000 questions. Watermarking makes no meaningful difference to 99,970 of them, but introduces a numerical error into the other 30.

The average score may barely change.

If the 30 changes are things like:

```
nice -> pleasant
```

nobody cares.

If they are medication quantities, legal deadlines, code, measurements, coordinates or identifiers, the average score tells us very little about the risk.

A normal benchmark asks:

> *Is the model still about as good overall?*

The question we actually need answered is:

> *How often does the watermark turn a correct answer into a wrong one, and what exactly did it change?*

That is a different test.

"Unbiased" does not mean "unchanged"

textGrain has another interesting property. OpenAI describes it as **unbiased**.

In normal English, "unbiased" can easily sound like the watermark does not change the model's behavior.

Here it has a more specific technical meaning.

In simple terms, textGrain is designed so that if we average many possible choices driven by the secret key, we get back the model's original next-token probabilities. [2]

Think about two coins.

One coin lands on heads 60% of the time. The other lands on heads 40% of the time.

Use both equally and the average becomes:

```
heads: 50%
tails: 50%
```

The average is balanced.

Each coin is still biased.

The textGrain paper says the same thing in mathematical terms. The guarantee is an average. For one specific key-driven choice, the probabilities used to select the token can be different from the model's original probabilities. [2]

The report also notes that keeping this balance at each individual step does not, by itself, prove that complete generated answers behave exactly like unwatermarked ones. [2]

This is not a side effect. The key has to affect token selection somehow, otherwise there is no watermark to detect. [2]

Getting the original distribution back on average is useful, but the user does not receive the average of many possible answers. The user receives one answer.

If one watermark-driven path produces `50 mg` where the normal generation produced `5 mg`, another possible path does not repair the answer the user already received.

We have not shown that textGrain makes this specific mistake. What we can say is that "unbiased" does not mean that every individual answer stays unchanged.

Watermarking needs room to work

Watermarks work better when the model has more choices.

OpenAI's own results show this.

A **false positive** is when the detector says normal text is watermarked even though it is not. At a false-positive rate of 1% - about one wrong accusation for every 100 normal texts - textGrain detected about 80% of its 200-token psychology passages and about 95% of the 400-token passages. [1]

Detection was substantially worse on mathematical text, where there is less freedom to choose different wording. [1]

Editing also damages the signal.

In OpenAI's tests, replacing 10% of the words in a 400-token passage with synonyms reduced detection from roughly 92% to 66%.

Replace 25%, and detection dropped to about 17%. [1]

So three things work against each other:

```
strong enough to detect
        /\
       /  \
      /    \
survives  does not
editing   change meaning
```

The watermark needs enough influence over the generated text to leave a pattern.

If it has almost no influence, there is little to detect.

But when the correct answer gives the model very little freedom, this influence becomes more dangerous.

And if somebody rewrites enough of the text, much of the watermark disappears anyway.

This is the trade-off: the watermark needs enough influence to be detected, but every bit of influence is another chance to push the answer away from what the model would normally produce.

Recipes are a surprisingly good example

Recipes sound trivial next to medical advice, but they show the problem clearly.

A recipe has lots of language that can change without changing the result:

```
Stir the mixture gently.
Mix the ingredients carefully.
```

These are good places to hide statistical choices.

A recipe also has values that must stay correct:

```
1 teaspoon
180 C
12 minutes
```

A tablespoon instead of a teaspoon can ruin dinner.

A gram instead of a milligram can cause a much bigger problem.

The issue is the same in both cases: **some tokens matter much more than others**.

An LLM does not mark every token as:

```
safe to change
```

or:

```
do not touch this
```

Entropy does not tell us this. Good grammar does not tell us this either.

If a watermark is going to influence token selection, it needs a better way to protect these values.

What do we actually need to test?

At this point, the question is no longer whether watermarking can make a correct answer worse. Research already shows that it can with several watermarking methods. [3][4]

For textGrain, the important question is how often this happens at production settings, and what changes when it does.

Testing this is not complicated.

Take questions where the normal model gets an exact fact right.

Generate the same answer with watermarking enabled.

Then look specifically at things that should not change:

- numbers;
- decimal points;
- units;
- dates and times;
- negation;
- medication names and quantities;
- legal references;
- account or case identifiers;
- URLs and hashes;
- code operators and literals;
- mathematical signs and exponents;
- structured data.

Do not only ask whether the two answers are "similar". Check whether the parts that must stay exact actually stayed exact.

Then separate two kinds of failure:

1. The watermark directly changed the factual token.

2. An earlier watermark choice changed the generation and the factual error happened later.

The second case matters because excluding numbers from watermarking does not solve it.

The results should also be reported like this:

```
correct without watermark -> correct with watermark
correct without watermark -> wrong with watermark
wrong without watermark   -> correct with watermark
wrong without watermark   -> wrong with watermark
```

The second line is the dangerous one, and it is also the one an average benchmark can easily hide.

Severity matters too.

Changing "nice" to "pleasant" and changing "mg" to "g" should not be counted as the same kind of difference.

OpenAI's public material gives us benchmark results, detection rates and mathematical guarantees.

What it does not give us is the number that matters here: how often did a correct answer become wrong because watermarking was enabled, and what changed?

That is the missing test.

Maybe the watermark should not be in the words

There is also a bigger question.

The EU AI Act requires providers of systems that generate synthetic text to mark the output in a **machine-readable** format - something software can detect automatically - and make it detectable as artificially generated or manipulated, as far as this is technically feasible. [9]

It does not require providers to do this by changing the words a model chooses. [9]

There are other ways to prove where content came from.

For example:

- a response can be digitally signed, so software can verify who produced it;
- an API provider can keep a signed or authenticated record of a response;
- a document can contain provenance metadata that describes how it was created;
- systems such as Content Credentials can attach signed information about the origin and editing history of media; [8]
- providers can keep audit logs that can later be used for verification.

These methods also have problems.

The biggest one is copy-and-paste. If text is copied out of a signed document, the signature stays with the document. It does not follow the copied paragraph.

This is why token-level watermarking is attractive: the signal stays inside the text.

But keeping the signal inside the text comes with a price: the answer itself becomes the storage space for the watermark.

For some types of output, keeping the answer exact may be more important than keeping the watermark attached after copy-and-paste.

So, are we enshittifying the answer?

Text watermarking is a clever solution to a real problem.

But it solves the provenance problem by adding another influence to the process that creates the answer.

The important question is not whether the text still looks natural. It is whether the watermark can turn a correct answer into a wrong one.

Recent research shows that this happens with several watermarking methods. [3][4]

We still do not know how often it happens with textGrain at production settings, especially in answers where changing one number, unit, operator or word like "not" can have a big effect.

If watermarking is going to be added to production LLM output at scale, this needs to be measured properly first.

Otherwise we may be enshittifying the answer in order to prove where the answer came from.

Sources

[1] OpenAI, Our approach to EU text provenance rules, October 5, 2026
https://openai.com/index/eu-text-provenance/

[2] Xiang Li et al., textGrain: Entropy-Calibrated Watermarking for Language Model Text, 2026
https://cdn.openai.com/pdf/e9508624-d767-41b6-a26d-e34ca798ada6/textgrain-entropy-calibrated-watermarking-for-language-model-text.pdf

[3] Haocheng Ye et al., Invisible Ink, Visible Lies: How Production Watermarking Causes LLMs to Hallucinate, accepted at NeurIPS 2026
https://arxiv.org/abs/2610.04860

[4] Rochana Prih Hastuti et al., Factuality Beyond Coherence: Evaluating LLM Watermarking Methods for Medical Texts, Findings of EMNLP 2025
https://aclanthology.org/2025.findings-emnlp.818/

[5] Taehyun Lee et al., Who Wrote this Code? Watermarking for Code Generation, ACL 2024
https://arxiv.org/abs/2305.15060

[6] John Kirchenbauer et al., A Watermark for Large Language Models, ICML 2023
https://proceedings.mlr.press/v202/kirchenbauer23a.html

[7] Sumanth Dathathri et al., Scalable watermarking for identifying large language model outputs, Nature 2024
https://www.nature.com/articles/s41586-024-08025-4

[8] Coalition for Content Provenance and Authenticity (C2PA), Content Credentials
https://c2pa.org/faqs/

[9] Regulation (EU) 2024/1689, Article 50
https://eur-lex.europa.eu/eli/reg/2024/1689/2026-07-27/eng